Claude Code'u iyi kullanmanın kontrol listesi

19 bölüm, 107 madde. Başlangıç için on madde, her maddede neden, nasıl, kopyalanabilir örnek ve doğrulama.

21 Ekim 2026 · Kaynak kontrolü; yerel CLI 2.1.284
Claude Code belgelerine dayanan teknik notlar ve editoryal iş akışı önerileri.

Nasıl okunur

Bu liste tekrar işi azaltmak, kapsamı belirlemek ve sonucu kanıtla teslim almak için hazırlanmış bir çalışma rehberidir. Her madde her projeye uygun değildir; kullanımına uygun olanları seç.

Neden gerekçeyi, **nasıl** uygulama yolunu, **doğrula** beklenen kanıtı gösterir. Başlangıç onlusu ve temel/ileri etiketleri editoryal öneridir; ölçülmüş getiri sıralaması değildir.

Temel 25, **ileri** 82 madde vardır. Teknik davranışların yanında resmî kaynak bağlantısı bulunur; bağlantısız maddeler iş akışı önerisidir. Örnekler kendi projenin araçlarına ve yetkisine uyarlanmalıdır.

Kaynaklar 21 Ekim 2026'da kontrol edildi. Yerel CLI 2.1.284 için yalnız sürüm ve yardım çıktısı alındı; listelenen bütün Claude özellikleri canlı çalıştırılmadı. CLI, Desktop, bulut ve sağlayıcı farklarını ilgili belgede kontrol et.

Bir kutuyu işaretlemek kişisel ilerleme kaydıdır; otomatik doğrulama veya güvenlik garantisi değildir. Tarayıcı verisi temizlenebilir; ilerlemeni JSON olarak dışa aktarabilirsin.

Önce bu on madde

Bütün listeyi okumaya vaktin yoksa buradan başla. Bunlar başlangıç önerileri; işine uymayanı atlayabilirsin.

1 **Bitiş ölçütünü baştan yaz** madde 3.1
Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

2 **Tutarlı oturum ve devir kaydı** madde 2.1
Birbirinden kopuk işleri aynı bağlama yığmak kararları karıştırabilir; her küçük adımda yeni oturum açmak da tekrar yaratır.

3 **Kısa ve doğrulanmış proje talimatı** madde 1.2
Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

4 **Projeye izin listesi** madde 6.1
Projeye ait dar liste tekrarlanan izinleri azaltabilir; test betikleri de kod çalıştırdığı için önce incelenmelidir.

5 **Gizli dosyaları okuma dışı bırak** madde 7.1
Gizli verinin bağlama veya çıktıya girmesini azaltmak gerekir; tek bir araç kuralı tam yalıtım sağlamaz.

6 **Alt ajan modelini ölçerek seç** madde 5.1
Daha küçük model her doğrulama işinde yeterli olmayabilir. Aynı temsilî işte kalite, süre ve kullanımı karşılaştır.

7 **"Geçti" yerine sayıyı oku** madde 12.1
Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

8 **Kaynak kodu Git'te, commit kapsamı belli** madde 15.1
Küçük, anlamlı commit geri almayı kolaylaştırır. Git dışı veriler ve sırlar için ayrıca uygun yedek gerekir.

9 **Düşünme bütçesini işe göre ayarla** madde 5.3
Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

10 **Eleştiriyi yazandan ayrı bağlama ver** madde 9.1
Ayrı bağlam farklı varsayımlarla inceleme yapabilir; bağımsızlık kalite garantisi değildir.

1 Talimat katmanları

Yeni oturumun konuşma bağlamı yenidir; talimat dosyaları ve otomatik bellek oturumlar arasında bilgi taşıyabilir. Kalıcı kuralları kısa tut, ayrıntılı prosedürleri ilgili işe taşınacak dosyalara ayır.

- [ ] **1.1 Global CLAUDE.md'ye yalnız varsayılan davranışı değiştiren kuralları yaz.** TEMEL

  NEDEN Talimatlar bağlamda yer kaplar. Gereksiz tekrar, ilgili kuralları bulmayı zorlaştırır.

  NASIL `~/.claude/CLAUDE.md` . Genel tavsiye değil, davranış yaz.

  ```
  # Davranışı tarif et
  - Yetkilendirilmiş değişiklikleri anlamlı commit'lere böl.
  - Push ve yayını kullanıcının belirttiği hedef ve kapsam içinde yap.
  - Karar sorarken önerini ve gerekçeni ekle.
  ```

  DOĞRULA Her kural için sor: silsem davranış değişir mi? Değişmiyorsa sil.

  Belge: Talimat ve bellek

- [ ] **1.2 Her aktif projede kısa ve doğrulanmış bir CLAUDE.md tut.** TEMEL

  NEDEN Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

  NASIL `/init` taslağını gözden geçir; gerçek test ve derleme komutlarını yaz. 100 satır bir ürün sınırı değildir. Uzun prosedürü skill veya kapsamlı kurala ayır.

  ```
  # Proje: fiyat-izleyici
  ## Komutlar
  - Test: npm test -- --run
  - Tek dosya: npm test -- src/parser.test.ts
  ## Tuzaklar
  - data/ altındaki JSON elle düzenlenmez, scripts/topla.mjs üretir.
  - Saat dilimi her yerde Europe/Istanbul.
  ```

  DOĞRULA Yeni oturumda dosya okutmadan "test komutu ne?" diye sor. Doğru cevap gelmeli.

  Belge: Talimat ve bellek

- [ ] **1.3 Makineye özgü bilgiyi `CLAUDE.local.md` 'ye yaz ve gitignore'la.** İLERİ

  NEDEN Repo başka makinede ya da bulut oturumunda açılınca yanlış yol talimatı kalmaz.

  NASIL Proje kökünde `CLAUDE.local.md` : venv yeri, yerel port, cihaz adı.

  DOĞRULA `git check-ignore CLAUDE.local.md` eşleşme döndürmeli.

  Belge: Talimat ve bellek

- [ ] **1.4 Yalnız belli dosyalarda geçerli kuralı `.claude/rules/` altına koy.** İLERİ

  NEDEN Kural yalnız eşleşen dosyaya dokunulunca yüklenir, geri kalan zamanda bağlamı doldurmaz.

  NASIL Frontmatter'da `paths` . Kullanıcı düzeyi için `~/.claude/rules/` .

  ```
  ---
  paths: ["**/*.swift"]
  ---
  - Görünüm dosyalarında iş mantığı yok, ViewModel'e taşı.
  - Yeni ekran eklenince Previews da eklenir.
  ```

  DOĞRULA Eşleşen bir dosyayı okut, ardından `/context` ile kuralın yüklendiğini gör.

  Belge: Talimat ve bellek

- [ ] **1.5 Dış dosya import'unun gerçekten yüklendiğini doğrula.** İLERİ

  NEDEN Proje dışı import onayı ve çalışma modu, hangi bilginin yüklendiğini etkileyebilir.

  NASIL Gerekli dosyayı `@path` ile belirt, dış import onayını yönet ve `/memory` listesinden kontrol et. SessionStart yalnız gerekli dinamik bilgiyi eklesin; onay atlatma yolu olarak kullanılmasın.

  DOĞRULA Etkileşimli ve headless kullanımını ayrı dene; yüklenmeyen bilgiyi varsayma.

  Belge: Talimat ve bellek

- [ ] **1.6 Başka bir ajanla paylaşılan repoda kuralları tek dosyada tut.** İLERİ

  NEDEN İki dosya ayrı ayrı güncellenince birbirinden kopar.

  NASIL Güncel sürüm AGENTS.md'yi doğrudan okuyabilir; aynı dizinde veya üstünde CLAUDE.md/CLAUDE.local.md bulunması varsayılan seçimi değiştirir. Ortak dosyayı uyumlu biçimde paylaşmak için aşağıdaki import kullanılabilir. `/memory` ile yüklenen dosyayı doğrula.

  ```
  # CLAUDE.md
  @AGENTS.md
  ```

  DOĞRULA Her iki araçta yüklenen dosya ve kritik kural aynı; benzer cevap tek başına yüklenme kanıtı değildir.

  Belge: Talimat ve bellek

- [ ] **1.7 Otomatik belleği düzenli olarak gözden geçir.** İLERİ

  NEDEN Talimat dosyası ile modelin biriktirdiği notlar farklıdır; eski çıkarım sonraki işi etkileyebilir.

  NASIL `/memory` ile kayıtları ve etkin ayarı incele; yanlış veya artık geçerli olmayan bilgiyi düzelt.

  DOĞRULA Yeni oturum kritik notu doğru kullanıyor; otomatik belleğe güvenilerek açık iş kaydı atlanmıyor.

  Belge: Talimat ve bellek

2 Bağlam yönetimi

Gereksiz araç çıktısı ve konu değişimleri bağlamı büyütür. Sıkıştırma ayrıntı kaybettirebilir; yeni oturum da bağlamı tekrar kurmayı gerektirir. Geçiş kararını işin sınırına ve açık kanıtlara göre ver.

- [ ] **2.1 Oturumu tutarlı bir iş etrafında tut; geçişte devir dosyası bırak.** TEMEL

  NEDEN Birbirinden kopuk işleri aynı bağlama yığmak kararları karıştırabilir; her küçük adımda yeni oturum açmak da tekrar yaratır.

  NASIL Oturum sonunda Claude'a proje kökünde bir devir dosyası yazdır, yeni oturum oradan başlasın.

  ```
  Oturumu kapatıyoruz. BEKLEYENLER.md'ye yaz:
  1. Sıradaki adım (komut, dosya, beklenen sonuç)
  2. Açık sorular
  3. Alınan kararlar ve gerekçeleri, denenip olmayanlar
  4. Bitenler (tarihli, kısa)
  ```

  DOĞRULA Yeni oturum açık işi, ilgili dosyayı ve son doğrulama sonucunu devirden bulabiliyor.

  Belge: İyi uygulamalar

- [ ] **2.2 Bağlamı tahminle değil sayıyla izle.** TEMEL

  NEDEN Doluluğu hissedemezsin. Sayı görünce devri zamanında yaparsın.

  NASIL `/context` ile anlık dağılım. `/statusline` ile alt satıra doluluk yüzdesi.

  ```
  /statusline model adını ve context_window.used_percentage değerini göster
  ```

  DOĞRULA Durum satırında yüzde görünüyor.

  Belge: Durum satırı

2.3 Konu değişince `/clear` de. TEMEL

NEDEN Eski konunun ayrıntısı yeni işte gürültüdür, yanlış varsayıma da yol açar.

NASIL Alakasız yeni iş için yeni oturum ya da `/clear`.

DOĞRULA Bir oturumda iki farklı projeye ait iş kalmamalı.

Belge: Komutlar

2.4 Çok dosya okuyan araştırmayı alt ajana ver. İLERİ

NEDEN Ham metin alt ajanın bağlamında kalır, ana oturuma yalnız sonuç döner.

NASIL Dönüş biçimini baştan söyle.

```
Bir alt ajanla src/ altındaki bütün ödeme akışını tara.
Bana yalnız şunları dön: giriş noktaları (dosya:satır),
harici servis çağrıları, hata yönetimi olmayan yerler.
```

DOĞRULA Ana oturumda dosya dökümü yerine kısa rapor görürsün.

Belge: Alt ajanlar

2.5 Büyük araç çıktısını dosyaya yazdır, bağlama yalnız özetini al. İLERİ

NEDEN Büyük araç çıktısı gerekli kanıtı görünmez kılabilir.

NASIL Tam çıktıyı göreve özel loga yaz; özeti ve komutun gerçek çıkış kodunu birlikte raporla.

```
mkdir -p work
npm test > work/test.log 2>&1
sonuc=$?
tail -n 30 work/test.log
exit "$sonuc"
```

DOĞRULA Sonda çalışan tail komutunun başarısı testin başarısı sanılmıyor.

Belge: Headless kullanım

2.6 `/compact` yaparken neyin korunacağını söyle. İLERİ

NEDEN Talimatsız sıkıştırma neyin önemli olduğunu tahmin eder.

NASIL Komutun arkasına korunacakları yaz.

```
/compact veri modeli kararlarını, açık hataları ve değişen dosya yollarını koru
```

DOĞRULA Sıkıştırmadan sonra kritik bir kararı sor, doğru gelmeli.

Belge: Komutlar

3 Görev verme

Bir isteğin kalitesi, en çok neyin "bitti" sayılacağının açıklığına bağlı. Bitiş tanımı yoksa model ya erken durur ya da senden "devam" bekler.

- [ ] **3.1 Bitiş ölçütünü baştan yaz.** TEMEL

  NEDEN Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

  NASIL Bitiş koşulunu ve durma sınırını yaz. `/goal` değerlendirmesi konuşmadaki kanıta bakar; bağımsız dosya veya test denetimi yapmaz. Metne yazılan tur sınırı da model tarafından yorumlanır, süreç düzeyinde katı bütçe değildir.

  ```
  Ödeme doğrulamasını uygula. Kabul: geçerli kart kabul, bozuk kart ret,
  servis hatasında tekrar deneme; ilgili testlerin sayısını ve çıktısını göster.
  /goal bu kabul senaryoları ve ilgili testler geçiyor; ya da 20 turda eksikleri
  raporlayıp dur
  ```

  DOĞRULA İstenen davranış ve test çıktısı görünür; sınırda duran iş tamamlandı diye raporlanmıyor.

  Belge: Hedefler

- [ ] **3.2 Kabul ölçütünü doğrulanabilir yaz.** TEMEL

  NEDEN "Çalışsın" denetlenemez, "şu test 12/12 geçsin" denetlenir.

  NASIL Sayı, ekran ya da komut çıktısıyla tarif et.

  ```
  Kötü: Arama düzgün çalışsın.
  İyi:  "ist" yazınca 300 ms içinde İstanbul ilk sırada gelir;
        boş sorguda liste temizlenir; search.test.ts 8/8 geçer.
  ```

  DOĞRULA Ölçütü başka biri okuyup evet/hayır diyebiliyor mu?

  İş akışı önerisi

- [ ] **3.3 Büyük ve belirsiz işte önce plan iste, küçük işte doğrudan yaptır.** TEMEL

  NEDEN Plan, yanlış yönü kod yazılmadan yakalar. Küçük işte plan zaman kaybıdır.

  NASIL Plan modu: CLI'da Shift+Tab ile modlar arasında geç ya da tek bir isteğin başına `/plan` yaz. Uygulamada mod seçiciyi kullan.

  DOĞRULA Plan aşamasında hedef uygulama dosyaları değişmemiş; izin isteyen bir araç plan modunda diye otomatik güvenli sayılmamış.

  Belge: Komutlar

- [ ] **3.4 Kısıtları ve yapılmayacakları açıkça söyle.** TEMEL

  NEDEN Model boşluğu en olası çözümle doldurur. O çözüm çoğu zaman senin istemediğindir.

  NASIL İsteğe ayrı bir "Kısıtlar" bloğu ekle.

  ```
  Kısıtlar:
  - Yeni bağımlılık ekleme.
  - Sunucu tarafı yok, veri cihazda kalır.
  - Mevcut renk ve boşluk değerlerini kullan.
  ```

  DOĞRULA Çıktıda kısıtı ihlal eden bir öğe var mı?

  İş akışı önerisi

- [ ] **3.5 İstediğin çıktının bir örneğini, mümkünse bir de karşı örneğini ver.** İLERİ

  NEDEN Bir örnek, bir paragraf tarifin anlatamadığı tonu ve biçimi taşır.

  NASIL "Şuna benzesin / şuna benzemesin" diye iki kısa örnek.

  ```
  Bildirim metni şuna benzesin: "Bugün 3 dakikan var mı? Bir sayfa kaldı."
  Şuna benzemesin: "Hedeflerinize ulaşmak için şimdi harekete geçin!"
  ```

  DOĞRULA İlk çıktı örneğe yakın mı?

  İş akışı önerisi

- [ ] **3.6 Claude'dan soru sorarken önerdiği cevabı da yazmasını iste.** İLERİ

  NEDEN Boş soru işi sana geri yükler. Önerili soruya "önerinle devam" demek yeter.

  NASIL Global CLAUDE.md'ye tek kural.

  ```
  - Sorduğun her soruya önerdiğin cevabı ve gerekçesini ekle.
  - Seçenekleri harflendir, tek mesajda tek karar iste.
  ```

  DOĞRULA Gelen sorulara tek kelimeyle cevap verebiliyor musun?

  İş akışı önerisi

- [ ] **3.7 Uzun bir brief'i yapıştırmadan önce eksiklerini buldur.** İLERİ

  NEDEN Brief'teki boşluk sonradan "beğenmedim" ve yeniden iş olarak döner.

  NASIL Kararı sonucu değiştiren boşlukları ve çelişkileri ayır; geri alınabilir uygulama ayrıntılarını öneriyle tamamla.

  ```
  Bu brief'te sonucu değiştiren belirsizlikleri ve çelişkileri listele.
  Her biri için öneri, gerekçe ve karar sahibini yaz.
  Geri alınabilir teknik ayrıntıları varsayım olarak belirtip ilerle.
  ```

  DOĞRULA Ürün davranışı veya dış taahhüt değiştiren kararlar çözülmüş; bağımsız işler gereksiz beklemiyor.

  İş akışı önerisi

4 Prompt yazımı: önce ve sonra

İyi bir istek uzun olmak zorunda değil. Dört şey taşıması yeter: amaç, "iyi"nin tanımı, bağlam ve çıktının biçimi. Aşağıdaki örnekler aynı isteğin zayıf ve güçlü hâlleri.

- [ ] **4.1 "En iyi" ya da "kaliteli" diyorsan ölçütünü yaz.** TEMEL

  NEDEN Tanımsız sıfatı model kendi ortalamasıyla doldurur.

  NASIL Sıfatın yerine iki-üç gözlemlenebilir özellik koy.

  ```
  Önce: Bana en iyi onboarding ekranını tasarla.
  Sonra: İlk açılışta 30 saniyede bitecek, en fazla 3 ekranlık,
         izin isteğini ilk değer görüldükten sonra soran bir onboarding.
         Üç farklı yön çiz, her biri tam ekran.
  ```

  DOĞRULA Ölçütü okuyan biri çıktıyı puanlayabilir mi?

  İş akışı önerisi

4.2 Okuru ve kullanım yerini söyle. TEMEL

NEDEN Aynı içerik yöneticiye, geliştiriciye ve müşteriye farklı yazılır.

NASIL "Kim okuyacak, nerede, ne karar verecek" cümlesi.

```
Önce: Bu sürüm için not yaz.
Sonra: Bu sürüm notunu App Store'daki "Yenilikler" alanı için yaz.
       Okur teknik olmayan kullanıcı. En fazla 4 madde, her biri
       kullanıcının fark edeceği bir değişiklik.
```

DOĞRULA Çıktının tonu ve ayrıntı düzeyi okura uyuyor mu?

İş akışı önerisi

4.3 Uzunluk istiyorsan dolguyu da yasakla. İLERİ

NEDEN "En az 10 sayfa" tek başına doldurma davetidir.

NASIL Uzunluk hedefinin yanına "madde uydurma" kısıtı.

```
Yeterli açıklama ve somut örnek ver. Hedef uzunluk yaklaşık 10 sayfa;
kaynak veya gerekçe yetmiyorsa kısa tut ve boşluğu açıkla.
Sayfa hedefi için tekrar, uydurma veri veya gereksiz bölüm ekleme.
```

DOĞRULA Silinse kimsenin aramayacağı bir paragraf var mı?

İş akışı önerisi

4.4 Çıktının biçimini şablonla ver. İLERİ

NEDEN Biçim belirsizse her bölüm farklı yapıda gelir, karşılaştırılamaz.

NASIL Her öğe için alan listesi.

```
Her madde şu biçimde:
- tek cümlelik eylem (fiille başlar)
- neden önemli (bir cümle)
- nasıl yapılır (komut, ayar ya da dosya)
- nasıl doğrularım
```

DOĞRULA Bütün öğeler aynı alanları taşıyor mu?

İş akışı önerisi

5 Model, effort ve maliyet

Her işe en güçlü modeli ve en yüksek düşünme bütçesini vermek limiti erken bitirir. Doğru eşleştirme, aynı limitle daha çok iş demek.

5.1 Alt ajan modelini işin zorluğu ve hata maliyetiyle seç. TEMEL

NEDEN Daha küçük model her doğrulama işinde yeterli olmayabilir. Aynı temsilî işte kalite, süre ve kullanımı karşılaştır.

NASIL Ajan tanımında `model` seç; genel varsayılan için aşağıdaki ayar kullanılabilir. Çağrı ve ajan tanımı bu varsayılanı geçersiz kılabilir; Explore/Plan da aynı varsayımı izlemez.

```
{
  "env": { "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet" }
}
```

DOĞRULA Çalışan ajanın gerçek modelini `/tasks` ile gör; sonuç kalitesini ayrı değerlendir.

Belge: Alt ajanlar

5.2 Model takma adı ile sürüme sabitleme arasında bilinçli seçim yap. TEMEL

NEDEN Takma ad güncellemeleri izler; tam model kimliği tekrar üretilebilirlik sağlar. Takma adın hedefi sağlayıcıya göre de değişir.

NASIL Gündelik işte hesabının desteklediği takma adı, karşılaştırmalı ölçümde tam kimliği kaydet.

```
{ "model": "opus" }
```

DOĞRULA `/model` ve oturum bilgisi beklediğin modeli gösteriyor; güncel takma ad en ucuz veya en iyi sonuç garantisi sayılmıyor.

Belge: Model ve effort

5.3 Düşünme bütçesini işe göre ayarla. TEMEL

NEDEN Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

NASIL Oturumda `/effort`, başlatırken `--effort` kullan. `max` oturum içindir; kalıcı `effortLevel` için desteklenen seviyeyi seç. Seçenekler modele ve sürüme bağlıdır.

```
/effort high
/effort status
claude --effort max
```

DOĞRULA `/effort status` ile seçimi gör; aynı kabul senaryosunda kalite ve süreyi karşılaştır.

Belge: Model ve effort

5.4 Kullanımı haftada bir kez sayıyla gör. İLERİ

NEDEN Limit bittiğinde değil, bitmeden önce neyin harcadığını bilmek gerekir.

NASIL `/usage` içinde oturum tüketimini ve varsa plan limitlerini ayrı oku. Tahmini dolar tutarı abonelik faturası değildir; API faturası için Console kayıtlarına bak.

DOĞRULA Notta dönem, cihaz kapsamı ve API/abonelik ayrımı var; yaklaşık yerel analiz tüm hesap kullanımı sayılmıyor.

Belge: Kullanım ve maliyet

5.5 Advisor'u zor kararlar için kullan. İLERİ

NEDEN İkinci modelin danışması da kullanım tüketir; API faturalaması ve abonelik limiti ayrı değerlendirilir.

NASIL `/advisor opus` ile aç, `/advisor off` ile kapat. Kullanılabilirlik model, sağlayıcı ve özellik erişimine bağlıdır; Anthropic API altyapısı kullanır, bu yalnız API anahtarı kullanıcıları demek değildir.

DOĞRULA `/advisor` ile bağlı modeli kontrol et.

Belge: Advisor

6 İzinler

İzin listesi tekrarlanan yetkili işlemleri kolaylaştırır. Etkin mod, yönetilen politika, hook ve sandbox da sonucu etkiler. Bir allow kuralı komutun her girdide güvenli olduğunu kanıtlamaz.

- [ ] **6.1 Her aktif projede `.claude/settings.json` izin listesi tut.** TEMEL

  NEDEN Projeye ait dar liste tekrarlanan izinleri azaltabilir; test betikleri de kod çalıştırdığı için önce incelenmelidir.

  NASIL Yalnız incelenmiş test ve derleme komutlarını ekle; yazma veya yayın komutunu işin mevcut yetkisine göre ayrı değerlendir.

  ```
  {
    "permissions": {
      "allow": ["Bash(npm test)", "Bash(npm run build)"]
    }
  }
  ```

  DOĞRULA İzin isteme davranışını zararsız bir denemeyle ve etkin modla kontrol et; bekleyen adım varsa nedeni kayıtlı.

  Belge: İzinler

- [ ] **6.2 Yıkıcı komutları deny listesine koy.** TEMEL

  NEDEN Normal izin değerlendirmesinde deny kuralları allow'dan önce gelir. Komut kalıpları tüm yıkıcı eşdeğerleri yakalamaz; mod/eklenti ve OS sınırlarını da incele.

  NASIL Global ya da proje ayarında `deny` dizisi.

  ```
  "deny": [
    "Bash(rm -rf:*)",
    "Bash(git push --force:*)",
    "Bash(git reset --hard:*)"
  ]
  ```

  DOĞRULA Gerçek veride yıkıcı komut çalıştırma. Geçici, verisiz bir test ortamında engellenen ve izinli karşı örneği kontrol et.

  Belge: İzinler

- [ ] **6.3 Allow girdisini dar tut.** İLERİ

  NEDEN Geniş bir joker, istemediğin alt komuta da izin verir.

  NASIL Araç adı yerine alt komut düzeyinde yaz.

  ```
  Geniş: "Bash(git:*)"            # push, reset, clean da dahil
  Dar:   "Bash(git status:*)", "Bash(git diff:*)", "Bash(git commit:*)"
  ```

  DOĞRULA Her joker için kapsadığı komutları ve seçenekleri gözden geçir; izinli betiğin içeriği değişince yeniden incele.

  Belge: İzinler

- [ ] **6.4 Komutu izin motorunun eşleştirebileceği biçimde yaz.** İLERİ

  NEDEN Kabuk genişletmeleri ve birleşik komutlar ek izin değerlendirmesine girebilir; doğrudan yorumlayıcı yolu davranışı sadeleştirir.

  NASIL venv'i aktive etme, yorumlayıcıyı doğrudan çağır.

  ```
  Karmaşık: source .venv/bin/activate && python -m pytest
  Daha kolay denetlenir: .venv/bin/python -m pytest
  ```

  DOĞRULA Allow'da olan bir komut yine soruyorsa içinde genişletme ara.

  Belge: İzinler

- [ ] **6.5 Tekrarlanan analizleri betiğe çevir.** İLERİ

  NEDEN Sürüm kontrollü betik, tekrar eden anlık koddan daha kolay incelenir. Betiği izin listesine almak içindeki her işlemi güvenli yapmaz.

  NASIL Analizi projede `scripts/` altına koy, betiği izin listesine ekle.

  DOĞRULA Betik değişiklikleri incelenmiş; çalıştırılan dosya ve sürüm raporda belli.

  Belge: İzinler

- [ ] **6.6 Geçmişten izin listesi önerisi al.** İLERİ

  NEDEN Hangi komutun sık sorulduğunu tahmin etmek yerine kayıttan çıkar.

  NASIL `/fewer-permission-prompts`.

  DOĞRULA Önerilen her girdiyi tek tek gözden geçir, yıkıcı olanı alma.

  Belge: İzinler

- [ ] **6.7 Etkin izin modunu başlangıçta kontrol et.** İLERİ

  NEDEN Plan, manuel, otomatik onay ve bypass farklı davranır.

  NASIL `/permissions` ve oturumun mod göstergesini oku; geniş bypass yerine görevine uygun izin/sandbox bileşimini seç.

  DOĞRULA Zararsız örnekte beklenen izin davranışı görülmüş; mod adı tek başına güvenlik kanıtı değil.

  Belge: İzinler

7 Güvenlik ve gizlilik

Araç izinleri, dış içerik ve veri paylaşımı farklı riskler taşır. Talimat, izin kuralı ve sandbox farklı katmanlardır; biri diğerinin yerine geçmez.

- [ ] **7.1 Gizli dosyaları okuma dışı bırak.** TEMEL

  NEDEN Gizli verinin bağlama veya çıktıya girmesini azaltmak gerekir; tek bir araç kuralı tam yalıtım sağlamaz.

  NASIL Read deny kuralları dosya araçlarını ve tanınan bazı kabuk erişimlerini kapsar; keyfî Python/Node alt süreçlerini bütünüyle engellemez. Güçlü ayrım için sırları ortamdan çıkar veya sandbox dosya erişim sınırlarını uygula.

  ```
  "deny": [
    "Read(./.env)",
    "Read(./.env.*)",
    "Read(./**/*.pem)",
    "Read(./secrets/**)"
  ]
  ```

  DOĞRULA Gerçek sır yerine sahte işaret içeren dosyayla dosya aracı ve alt süreç sınırlarını ayrı test et.

  Belge: İzinler

- [ ] **7.2 Kurumsal veriyi kişisel hesaba ve depoya taşıma.** TEMEL

  NEDEN Şirket içi adresler, müşteri verisi ve iç doküman, özel bir depoda bile şirketin dışına çıkmış olur.

  NASIL Şirket işini şirketin onayladığı hesapla yap. Paylaşmadan önce iç alan adı ve iç kod deposu adresleri için tarama yaptır.

  ```
  Bu klasörü paylaşmadan önce tara: şirket alan adları, iç git adresleri,
  IP adresleri, e-posta adresleri, token'a benzeyen diziler.
  Bulduklarını dosya:satır olarak listele, hiçbirini silme.
  ```

  DOĞRULA Push öncesi tarama boş dönüyor.

  Belge: Güvenlik

- [ ] **7.3 Web sayfası, e-posta ve belgedeki talimata uyma refleksini kapat.** TEMEL

  NEDEN Okunan içerikteki "şunu yap" cümlesi senin talimatın değildir. Prompt injection böyle çalışır.

  NASIL Dış içerikle çalışırken izin modunu sıkı tut, yan etkili adımları (gönder, sil, öde) onaya bağla.

  DOĞRULA Dış metindeki talimat yetki sayılmıyor; gönderme, silme veya ödeme yalnız kullanıcının geçerli yetkisi kapsamındaysa uygulanıyor.

  Belge: Güvenlik

- [ ] **7.4 Ağ erişimini içerik güveniyle karıştırma.** İLERİ

  NEDEN Bir alan adına veya curl komutuna izin vermek indirilen içeriği güvenilir talimata dönüştürmez.

  NASIL Gereken adres ve araçla sınırla, indirilen betiği çalıştırmadan incele. WebFetch, Bash ve MCP erişimlerini ayrı değerlendir.

  DOĞRULA Dış içerikteki talimat uygulama yetkisi olarak kabul edilmiyor; erişim kapsamı kayıtlı.

  Belge: Güvenlik

- [ ] **7.5 Güvenmediğin kodda sandbox'ı aç.** İLERİ

  NEDEN İndirilen bir repodaki betik dosya sistemine ve ağa serbestçe erişmesin.

  NASIL `/sandbox` ile destek ve etkin sınırları incele. Varsayılan okuma erişimi geniş olabilir; gerekli denyRead sınırlarını, ağ alanlarını ve sandbox dışı istisnaları kontrol et. Yerleşik Read/WebFetch araçları ayrıca izin kurallarına tabidir.

  ```
  { "sandbox": { "enabled": true } }
  ```

  DOĞRULA İzinli ve yasaklı geçici dosya yollarında zararsız deneme yap; yalnız enabled=true görmeyi yalıtım kanıtı sayma.

  Belge: Sandbox

- [ ] **7.6 Tehlikeli komutu hook'la içeriğinden yakala.** İLERİ

  NEDEN Hook politika kontrolünü araç çağrısından önce çalıştırabilir; basit metin araması tüm komut eşdeğerlerini yakalamaz.

  NASIL PreToolUse engellemesi için çıkış kodu 2 kullan. Girdiyi doğrula, hatada engelleme davranışını ve timeout'u test et. Aşağıdaki örnek yalnız öğretici bir kalıp kontrolüdür.

  ```
  #!/bin/sh
  # Öğretici örnek; kapsamlı bir komut güvenlik motoru değildir.
  komut=$(jq -er ' .tool_input.command | select(type == "string") ' ) || {
    echo "Geçersiz hook girdisi" >&2; exit 2;
  }
  case "$komut" in
    *"DROP TABLE"*|*"--force"*) echo "Politika denetimi engelledi" >&2; exit 2;;
  esac
  exit 0
  ```

  DOĞRULA Sahte girdide izinli, engelli ve bozuk JSON vakaları test edilmiş; gerçek komutu çalıştırmadan sonuç görülüyor.

  Belge: Hook referansı

- [ ] **7.7 Yeni depodaki hook ve MCP yapılandırmasını güvenmeden önce incele.** İLERİ

  NEDEN Proje ayarları yerel komut veya dış araç başlatabilir.

  NASIL Settings, hook betikleri ve MCP sunucu komutlarını oku; paket kaynağını ve istenen yetkileri kontrol et.

  DOĞRULA Bilinmeyen komut otomatik çalıştırılmamış; güven kararı incelenen sürüme ait.

  Belge: Ayarlar

8 Hook'lar

Hook belirli olaylarda çalışan bir mekanizmadır; güvenilirliği betiğin kapsamı, izinleri, hata davranışı ve testleriyle sınırlıdır.

- [ ] **8.1 Her oturuma girmesi gereken bilgiyi SessionStart hook'una koy.** İLERİ

  NEDEN Talimatı model unutabilir. Hook her oturumda çalışır.

  NASIL Betiğin stdout'u oturumun bağlamına girer.

  ```
  "hooks": {
    "SessionStart": [{ "hooks": [{
      "type": "command",
      "command": "head -80 BEKLEYENLER.md 2>/dev/null || true"
    }]}]
  }
  ```

  DOĞRULA Okuma araçları kapalı bir `claude -p` ile içerikten soru sor.

  Belge: Hook referansı

8.2 Biçimlendirmeyi düzenleme sonrası hook'a bağla. İLERİ

NEDEN Biçimi her seferinde istemek yerine her düzenlemeden sonra kendiliğinden düzelir.

NASIL PostToolUse içinde yalnız değişen, desteklenen dosyayı biçimlendiren bir betik çağır. Betik JSON girdisindeki yolu doğrulasın ve proje dışına çıkmasın; tüm depoyu her düzenlemede taramasın.

```
"PostToolUse": [{
  "matcher": "Edit|Write",
  "hooks": [{ "type": "command",
    "command": "python3 .claude/hooks/format_changed.py" }]
}]
# format_changed.py projene göre yazılmalı ve test edilmelidir.
```

DOĞRULA Bozuk girintili bir dosya düzenlet, sonuç biçimli gelmeli.

Belge: Hook referansı

8.3 Bağlam göstergesini belgelenmiş statusline alanından üret. İLERİ

NEDEN Transcript iç yapısını token sayacı gibi ayrıştırmak sürüm değişiminde bozulabilir.

NASIL Statusline JSON girdisindeki `context_window.used_percentage` alanını kullan; ilk yanıt öncesi eksik/null değerini işle.

DOĞRULA Sıfır, null ve yüksek doluluk örneklerinde gösterge doğru; bilinmeyen değer yüzde sıfır diye sunulmuyor.

Belge: Durum satırı

8.4 Hook'u test et ve bozarak testin düştüğünü gör. İLERİ

NEDEN Hiçbir şeye bakmayan hook da sessizce "çalışır".

NASIL Hook'a örnek bir JSON girdisi verip çıktısını kontrol eden küçük bir test.

```
printf '%s\n' '{"cwd":"/tmp/ornek-proje"}' | sh .claude/hooks/oturum-basi.sh
# Test: izinli, engelli ve bozuk JSON girdilerini ayrı doğrula.
```

DOĞRULA Hook'u bilerek boz, test düşmeli.

Belge: Hook referansı

8.5 Hook hata politikasını görevine göre seç. İLERİ

NEDEN Bilgi gösteren hook ile güvenlik kapısı aynı hatada-devam kuralını kullanmamalıdır.

NASIL Bilgilendirme hatasını görünür kaydet; politika hook'unda hatayı sessizce başarıya çevirme. `onFailure: "block"` komut/HTTP hook'ları için v2.1.295+ gerektirir; eski sürümde yokmuş gibi davran.

DOĞRULA Betiğin bulunamaması, bozuk JSON, hata çıkışı ve timeout ayrı denenmiş; engel gerektiğinde çağrı ilerlemiyor.

Belge: Hook referansı

9 Skill, komut, ajan, MCP

Aynı tarifi üçüncü kez yazıyorsan, o tarif bir dosyaya aittir. Skill gerektiğinde yüklenir, ajan kendi bağlamında çalışır, MCP dış sisteme API ile bağlanır.

- [ ] **9.1 Eleştiriyi yazandan ayrı bir bağlama ver.** TEMEL

  NEDEN Ayrı bağlam farklı varsayımlarla inceleme yapabilir; bağımsızlık kalite garantisi değildir.

  NASIL Statik inceleme için yazma ve Bash aracı olmayan doğrulayıcı ajan kullan. Test çalıştırmak gerekiyorsa ayrı, sınırlı ortam ve izin ver; Bash olan ajan salt okunur sayılmaz.

```
---
name: dogrulayici
description: Tamamlanan değişiklikte kanıtlı doğruluk hatalarını statik olarak
inceler.
tools: Read, Grep, Glob
model: sonnet
---
İddiaları diff ve kaynakla karşılaştır. Bulgu uydurma.
Her bulguda dosya, tetikleyici ve etkisini yaz. Test çalıştırmadıysan belirt.
```

  DOĞRULA Bulgu varsa dosya ve kanıtı var; bulgu yoksa kapsam ve çalıştırılmayan kontroller açık.

  Belge: Alt ajanlar

- [ ] **9.2 Üçüncü kez anlattığın işi skill'e çevir.** İLERİ

  NEDEN Tarif her seferinde biraz farklı anlatılır, sonuç da farklı çıkar.

  NASIL `~/.claude/skills/<ad>/SKILL.md` .

```
---
name: surum-notu
description: Son etiketten bu yana değişikliklerden kullanıcıya dönük sürüm notu
yazar. "Sürüm notu", "release notes" denince kullan. Teknik changelog için kullanma.
---
1. git log ile son etiketten bu yanaki commit'leri al.
2. Kullanıcının fark edeceği değişiklikleri seç, en fazla 5.
3. Her maddeyi kullanıcının diliyle bir cümlede yaz.
```

  DOĞRULA Açık /surum-notu çağrısı ve doğal dille eşleşen istek ayrı denenmiş; otomatik seçilmeme de kaydedilmiş.

  Belge: Skills

- [ ] **9.3 Skill açıklamasına "kullanma" sınırı koy.** İLERİ

  NEDEN Model skill'i açıklamadan seçer. Sınır yoksa komşu işlerde de yüklenir.

  NASIL Açıklamaya hem tetikleyici ifadeleri hem de kullanılmayacağı durumu yaz (yukarıdaki örnekteki son cümle).

  DOĞRULA Benzer ama farklı bir istekte skill yüklenmemeli.

  Belge: Skills

- [ ] **9.4 Yan etkili skill'i kendiliğinden çalışmaya kapat.** İLERİ

  NEDEN Deploy ya da mesaj gönderen bir skill yanlış eşleşmede çalışmasın.

  NASIL Frontmatter'da `disable-model-invocation: true` otomatik çağrıyı kapatır; araç izinleri veya kullanıcı yetkisi yerine geçmez. Etkin skill override ayarını da kontrol et.

```
disable-model-invocation: true
```

  DOĞRULA İlgili cümleyi yaz, skill kendiliğinden çalışmamalı.

  Belge: Skills

- [ ] **9.5 Bir uygulamaya bağlanırken önce o uygulamanın MCP'sini dene.** İLERİ

  NEDEN API tabanlı araç hızlı ve kesin. Ekranda tıklayarak sürmek yavaş ve kırılgan.

  NASIL Sıra: uygulamanın MCP'si, sonra tarayıcı, en son bilgisayar kontrolü.

  DOĞRULA Aynı iş için kaç araç çağrısı gittiğine bak.

  Belge: MCP

- [ ] **9.6 Kendi setini plugin olarak paketle.** İLERİ

  NEDEN Başka makineye ya da ekip arkadaşına tek komutla taşınır.

  NASIL `.claude-plugin/plugin.json` ve skill dizinleri.

  DOĞRULA Temiz bir makinede kurulumdan sonra skill listede görünmeli.

  Belge: Plugins

- [ ] **9.7 MCP bağlantısına yalnız gereken kapsamı ver.** İLERİ

  NEDEN Bağlantının çalışması doğru hesap ve en az yetki anlamına gelmez.

  NASIL `/mcp` ile sunucu ve hesap durumunu incele; okuma/yazma yetkisini ayır, gereksiz bağlantıyı kaldır.

  DOĞRULA Zararsız okuma testi doğru hedefte; yazma ayrıca kullanıcının yetki kapsamıyla eşleşiyor.

  Belge: MCP

- [ ] **9.8 Skill'i olumlu ve olumsuz örneklerle değerlendir.** İLERİ

  NEDEN Tek başarılı çağrı yanlış eşleşmeleri ve eksik çıktıları yakalamaz.

  NASIL Tetiklenmesi gereken, gerekmeyen ve hata durumunu içeren küçük vaka seti tut; beklenen sonucu baştan yaz.

  DOĞRULA Gerekli çağrıda doğru çıktı, komşu istekte gereksiz çağrı yok; başarısız vaka saklanmış.

  Belge: Skills

10 Ekiple kullanım

Bir kişinin iyi düzeni, ekip için de düzen olabilir. Ama ortak dosyaya giren her şey herkesin oturumunu etkiler.

- [ ] **10.1 Ortak ayarları `.claude/settings.json`'da, kişisel olanları `settings.local.json`'da tut.** İLERİ

  NEDEN Ekibin izin listesi ve hook'ları git ile paylaşılır, kişisel tercihin başkasına geçmez.

  NASIL `settings.json` commit'lenir, `settings.local.json` git dışında kalır. Öncelik sırası yukarıdan aşağı: yönetilen ayarlar, komut satırı, `settings.local.json`, `settings.json`, kullanıcı ayarları.

  DOĞRULA Yeni ekip üyesi dosyaları ve etkin politikayı doğruluyor; yönetilen ayar veya kişisel mod farkı aynı izin garantisi sayılmıyor.

  Belge: Ayarlar

- [ ] **10.2 Proje CLAUDE.md'sini kod gibi gözden geçir.** İLERİ

  NEDEN Bir kural bütün ekibin oturumunu değiştirir.

  NASIL CLAUDE.md değişikliği de pull request'ten geçsin.

  DOĞRULA Son ay eklenen kuralların gerekçesi commit mesajında yazıyor mu?

  Belge: Talimat ve bellek

- [ ] **10.3 Ekipte tekrar eden işi ortak skill ya da plugin yap.** İLERİ

  NEDEN Herkes aynı işi kendi tarifinle yaptırınca sonuçlar tutarsız olur.

  NASIL Repoda `.claude/skills/` ya da ekip plugin'i.

  DOĞRULA İki kişinin aynı skill ile ürettiği çıktı aynı biçimde.

  Belge: Skills

11 Paralel çalışma

Aynı anda çok oturum çalıştırmak güçlü bir kaldıraç. Ama her paylaşılan kaynak bir çakışma noktası ve makine yükü her şeyi yavaşlatır.

- [ ] **11.1 Aynı dosyalara iki yazar sokma. Her yazar ayrı worktree'de.** İLERİ

  NEDEN İki oturum aynı dosyayı düzenlerse biri diğerinin işini ezer.

  NASIL Worktree ile başlat.

  ```
  claude --worktree odeme-ekrani
  ```

  DOĞRULA `git worktree list` 'te her yazar için ayrı satır.

  Belge: İş akışları

- [ ] **11.2 Git index'ini ortak yazarlardan ayır ve commit kapsamını incele.** İLERİ

  NEDEN Aynı checkout index'i ortaktır; ayrı worktree index'leri ayrıdır. İki komutu && ile birleştirmek atomik kilit sağlamaz.

  NASIL Ayrı worktree kullan, yalnız kendi değişikliklerini stage et ve commit öncesi staged diff'i incele.

  ```
  git status --short
  git add -- src/odeme.ts
  git diff --cached -- src/odeme.ts
  git commit -m "Ödeme: kart doğrulama"
  # Önceden stage edilmiş başka dosya varsa önce kapsamı ayır.
  ```

  DOĞRULA Commit diff'inde başka işin değişikliği yok.

  Belge: Git worktree

- [ ] **11.3 Port ve tarayıcı oturumunu projeye özel yap.** İLERİ

  NEDEN Aynı porttaki iki sunucu birbirini düşürür, yanlış sayfa test edilir.

  NASIL Her projeye sabit ve farklı bir port.

  DOĞRULA Açılan sayfa gerçekten bu projenin sayfası.

  İş akışı önerisi

- [ ] **11.4 Ağır oturum sayısını makine yüküne göre sınırla.** İLERİ

  NEDEN Kaynak baskısı yavaşlama ve kararsız testlerle birlikte görülebilir; tek yük ölçümü nedeni kanıtlamaz.

  NASIL CPU, bellek baskısı, disk ve aktif işleri birlikte izle; bağımsız ağır işi kaynaklara göre sırala.

  DOĞRULA Bekleme süresi ve başarısızlıklar ölçülmüş; sabit bir load-average eşiği her makineye uygulanmıyor.

  İş akışı önerisi

- [ ] **11.5 Uzun gözetimsiz işi bulut oturumuna gönder.** İLERİ

  NEDEN Makineyi yormaz, uyku onu öldürmez.

  NASIL Uygulamada bulut hedefli oturum. Yerel cihaz isteyen iş yerelde kalır.

  DOĞRULA Bilgisayar kapalıyken iş ilerlemiş olmalı.

  İş akışı önerisi

- [ ] **11.6 Çok ajanlı iş akışını yalnız gerçekten paralel işe kullan.** İLERİ

  NEDEN Onlarca ajan çok token harcar. Sıralı iş paralelden kazanmaz.

  NASIL Bağımsız parçalar için, örneğin on dosyanın ayrı ayrı incelenmesi.

  DOĞRULA İş akışı sonrası kullanım farkına `/usage` 'dan bak.

  Belge: Alt ajanlar

12 Doğrulama

"Geçti" demek bir iddia, kanıt değil. En tehlikeli yeşil, hiçbir şeye bakmadığı için yeşil olandır.

- [ ] **12.1 Sonuçta durumu değil sayıyı oku.** TEMEL

  NEDEN Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

  NASIL Raporda test sayısını iste.

  ```
  Yeşil görünen ama boş:  Test run with 0 tests ... SUCCEEDED
  Gerçek:                 Executed 42 tests, with 0 failures
  ```

  DOĞRULA Her "geçti"nin yanında bir sayı.

  İş akışı önerisi

- [ ] **12.2 Yeni kontrolü hatayı geri koyarak doğrula.** TEMEL

  NEDEN Kontrol gerçekten bakıyorsa bozuk kodda düşmeli.

  NASIL İzole fixture veya geçici branch'te kusuru geri getir; testin düştüğünü ve düzeltilmiş eşinin geçtiğini göster. Kullanıcının çalışma dosyasını koru.

  ```
  Yazdığın testin gerçekten çalıştığını kanıtla: fiyat hesabındaki
  indirimi geçici olarak kaldır, testi koş, düştüğünü göster, sonra geri al.
  ```

  DOĞRULA Bozuk sürümde kırmızı.

  İş akışı önerisi

- [ ] **12.3 Alt ajanın raporunu kanıtla karşılaştır.** TEMEL

  NEDEN Ajan raporu bir iddiadır.

  NASIL Diff'e, dosyaya, çıktıya bak. Ondan sonra "bitti" de.

  DOĞRULA Rapordaki bir iddiayı rastgele seç ve doğrula.

  İş akışı önerisi

- [ ] **12.4 İş sonunda üç satır istet: kaçırılan, en zayıf iddia, gerekçesiz karar.** İLERİ

  NEDEN Övgü değil kırma denemesi, sorunun kendisini sana getirir.

  NASIL CLAUDE.md'ye tek kural ya da bir slash komutu.

  ```
  İş bitince sormadan üç satır yaz:
  1. Neyi kaçırmış olabilirim (bakmadığım dosya, koşmadığım yol)
  2. Hangi iddiam en zayıf
  3. Hangi kararım gerekçesiz
  ```

  DOĞRULA Üç satırın her biri somut bir dosya ya da karar adı içeriyor mu?

  İş akışı önerisi

- [ ] **12.5 Görsel değişikliği önce/sonra ekran görüntüsüyle gör.** İLERİ

  NEDEN "Ne değişti" sorusu metinle cevaplanmaz.

  NASIL İki ekran görüntüsü yan yana.

  DOĞRULA Farkı ilk bakışta görebiliyor musun?

  İş akışı önerisi

- [ ] **12.6 Önemli değişikliği teslimden önce kod incelemesinden geçir.** İLERİ

  NEDEN Yazan bağlam kendi hatasını kaçırır.

  NASIL `/code-review` ile hedef diff'i incele. `--fix` değişiklik yapar, `--comment` dışarı yorum gönderir; bunları amaç ve yetki olmadan ekleme.

  DOĞRULA Bulgu ve kanıtları veya kapsamı belirtilmiş “bulgu yok” sonucu var; test yapılmadıysa ayrı yazıyor.

  Belge: Komutlar

- [ ] **12.7 Teslim aşamasını kanıtıyla adlandır.** İLERİ

  NEDEN Yerel test, push ve herkese açık yayın farklı sonuçlardır.

  NASIL PASS, FAIL, NOT_RUN ve BLOCKED durumlarını ilgili komut, tarih ve sürümle kaydet; dış hedefi tekrar oku.

  DOĞRULA Build geçti diye cihaz testi, upload oldu diye canlı yayın geçti denmiyor.

  İş akışı önerisi

13 Gözetimsiz koşum

"Ben yatıyorum, sabaha bitir" dediğin iş, sabah çalışan bir sonuç bırakmalı. Gece boyu izin ekranında beklemek ya da sessizce ölmek en pahalı başarısızlık.

- [ ] **13.1 Gözetimsiz koşum için talimat setini önceden yaz.** İLERİ

  NEDEN "Sormadan ilerle" demek, hangi durumda ne yapılacağını söylemez.

  NASIL İşe özel yetki, kapsam, bütçe ve engel davranışını belirt; her proje için sınırsız global yetki yazma.

  ```
  ## Bu görevde ben yokken
  - Yetkili kapsam içindeki geri alınabilir kararları ver, varsayımı yaz.
  - Eksik yetki veya sonucu değiştiren karar varsa ilgili adımı engelli işaretle.
  - Bağımsız işleri tamamla; başarısız testi gizleme.
  - Sonuç: yapılan, yapılmayan, kanıt, kalan karar ve kaynak temizliği.
  ```

  DOĞRULA Bir koşumu dene, sabah rapor eksiksiz mi?

  İş akışı önerisi

- [ ] **13.2 Ayrılmadan önce izin, disk, yük ve uyku koşullarını doğrula.** İLERİ

  NEDEN Bu dördünden biri geceyi boşa çıkarır.

  NASIL İzinleri etkin ayardan, kaynakları işletim sistemi araçlarından kontrol et. macOS örneği aşağıda; diğer sistemlerde eşdeğerini kullan.

  ```
  df -h .
  uptime
  pmset -g
  # Claude oturumunda /permissions ile etkin izinleri incele.
  ```

  DOĞRULA Dört kontrolün sonuçları ve sınırları yazılmış; ilk 20 ayar satırını okumak bütün izin denetimi sayılmıyor.

  İş akışı önerisi

- [ ] **13.3 Uzun işi kaldığı yerden devam edecek biçimde kur.** İLERİ

  NEDEN Uyku ya da çökme işi baştan başlatmasın.

  NASIL Ara çıktıyı diske yaz, her adım öncesi "bu adım bitti mi" kontrolü.

  DOĞRULA İşi yarıda öldür, yeniden başlat, tamamlanan adımlar atlanmalı.

  İş akışı önerisi

- [ ] **13.4 Zamanlanmış oturumda uzun işi arka plana atıp turu kapatma.** İLERİ

  NEDEN Arka plan süreci oturumla birlikte ölebilir ve geriye log bile kalmayabilir.

  NASIL Turu açık tut, log'u oturumdan bağımsız bir yola yaz.

  DOĞRULA Sabah log'un son satırı işin bitişini gösteriyor mu?

  Belge: Masaüstü

- [ ] **13.5 İş bitince açık kaynakları kapat.** İLERİ

  NEDEN Açık kalan simülatör, sunucu ya da tarayıcı belleği ve diski tüketir.

  NASIL Koşum sonunda kendi açtığın süreçleri kapat, başka oturumunkine dokunma.

  DOĞRULA Sabah yalnız beklediğin süreçler açık.

  İş akışı önerisi

- [ ] **13.6 Geçici derleme ve bağımlılıkları görev sonunda hedefli temizle.** İLERİ

  NEDEN Göreve özel ürün ve cache diskte birikir; isim benzerliği sahiplik kanıtı değildir.

  NASIL Başta kesin yolları kaydet. Kullanım bitince logları ve teslim dosyalarını koruyup yalnız göreve ait yeniden üretilebilir artıkları kaldır.

  DOĞRULA Silinen yol yok; kaynak, release arşivi, sonuç ve paylaşılan cache korunmuş.

  İş akışı önerisi

14 Rutinler ve zamanlama

Tekrarlanan işi elle tetiklemek unutulur. Ama bilgisayar kapanınca ölen bir rutin de güvenilir değildir.

- [ ] **14.1 Zamanlamanın nerede çalıştığını açıkça seç.** İLERİ

  NEDEN Bulut görevleri bilgisayarından bağımsızdır; yerel Desktop işi cihaz ve uygulama koşullarına bağlıdır.

  NASIL Bulut görevleri için `/schedule` ve desteklenen claude.ai hesabını kontrol et. Araştırma önizlemesinin koşulları değişebilir; Desktop ve oturum içi döngüyü aynı mekanizma sanma.

  DOĞRULA Doğru ortamda bir deneme koşumu, sonraki çalışma zamanı ve sonuç bağlantısı görülmüş.

  Belge: Bulut zamanlama

14.2 Headless rutinde çıktıyı ve tur sayısını sınırla. İLERİ

NEDEN Sınırsız koşum takılı kalınca saatlerce harcar.

NASIL JSON çıktı, tur sınırı, kendi log yolu.

```
mkdir -p work/runs
claude -p "$(cat gorev.md)" --output-format json --max-turns 30 \
  > work/runs/sonuc.json 2> work/runs/hata.log
sonuc=$?
printf '%s\n' "$sonuc" > work/runs/exit-code.txt
exit "$sonuc"
```

DOĞRULA JSON parse ediliyor, hata/başarı alanı ve süreç çıkış kodu birlikte kaydediliyor; limit dolması başarı değil.

Belge: Headless kullanım

14.3 Rutinin sessiz ölümünü ayrı bir nöbetle yakala. İLERİ

NEDEN Hiç koşmayan rutin hata da vermez.

NASIL Son başarılı koşumun zamanını kontrol eden ikinci küçük bir rutin.

DOĞRULA Bir rutini bilerek durdur, nöbet uyarmalı.

İş akışı önerisi

14.4 Dış servis kotasına uy ve yeniden denemeyi sınırla. İLERİ

NEDEN Eşzamanlı istekler kotayı tüketebilir; zamanları rastgele dağıtmak erişim engelini aşma yöntemi değildir.

NASIL Servis koşullarına ve Retry-After yanıtına uy; sınırlı geri çekilme ve jitter uygula.

DOĞRULA 429/503 denemesinde bekleme ve azami deneme sayısı doğru; engelde görev durumu açık.

İş akışı önerisi

14.5 Oturum içi /loop ile kalıcı zamanlamayı ayır. İLERİ

NEDEN /loop çalışan oturumun yaşam döngüsüne bağlıdır; cihazdan bağımsız bulut görevi değildir.

NASIL `/loop 5m` gibi sabit aralıklı döngünün görev kaydını ve durdurma biçimini kontrol et; bilgisayardan bağımsız işte bulut zamanlamasını seç.

```
/loop 5m Yayının tamamlanıp tamamlanmadığını kontrol et;
anlamlı değişiklikte bildir, tamamlanınca döngüyü iptal et.
```

DOĞRULA Bir deneme tetiklenmiş, iptal sonrası yeni koşum yok; oturum kapalıyken çalışma varsayılmıyor.

Belge: Oturum içi zamanlama

14.6 Headless başarısını süreç ve sonuç alanından birlikte doğrula. İLERİ

NEDEN Geçerli JSON üretmek işin tamamlandığını göstermez.

NASIL Çıkış kodu, `is_error`, sonuç türü ve kabul ölçütünü değerlendir; yarım işi tekrar koşarken yan etkileri çoğaltma.

DOĞRULA İzin reddi ve tur sınırı vakaları hata/eksik olarak görünür; başarılı eşinde kabul ölçütü sağlanır.

Belge: Headless kullanım

15 Git ve yayın

Git, Claude ile çalışırken en güvenilir geri alma düğmesi. Sık commit, Claude'a cesur değişiklik yaptırmanın güvencesi.

- [ ] **15.1 Kaynak kodunu Git'te tut ve commit kapsamını tanımla.** TEMEL

  NEDEN Küçük, anlamlı commit geri almayı kolaylaştırır. Git dışı veriler ve sırlar için ayrıca uygun yedek gerekir.

  NASIL `git init`, sonra CLAUDE.md'ye commit kuralı.

  ```
  - Sormadan commit at, mantıklı parçalara böl.
  - Push ve deploy yalnız açıkça istendiğinde.
  ```

  DOĞRULA `git log --oneline`'da her satır tek bir değişikliği anlatıyor.

  Belge: Git worktree

- [ ] **15.2 Checkpoint'in sınırını bil.** TEMEL

  NEDEN `/rewind` yalnız dosya araçlarıyla yapılan düzenlemeleri geri alır. Bash'le silinen ya da taşınanı almaz.

  NASIL Bash'le toplu değişiklikten önce commit.

  DOĞRULA Commit öncesi diff incelenmiş; Bash, dış servis ve veritabanı etkileri için ayrı geri alma yolu var.

  Belge: Checkpoint

- [ ] **15.3 Push ve deploy yetkisini hedefe ve kapsama bağla.** İLERİ

  NEDEN Uzak depo ve yayın hedefi yerel değişiklikten farklı bir teslim aşamasıdır.

  NASIL Kullanıcının mevcut yayın talimatını koru; belirsiz hedef veya yeni dış taahhütte karar iste. Geniş, kalıcı push/deploy iznini varsayılan yapma.

  DOĞRULA Doğru hesap, repo, dal ve görünürlük doğrulanmış; yayından sonra canlı sonuç okunmuş.

  Belge: İzinler

- [ ] **15.4 Commit atfını talimatla değil ayarla yönet.** İLERİ

  NEDEN Talimat unutulabilir, ayar her commit'te uygulanır.

  NASIL `attribution` ayarı (`includeCoAuthoredBy` eskidi). Boş metin o atfı gizler, `false` hepsini gizler (v2.1.281 ve sonrası).

  ```
  { "attribution": { "commit": "", "pr": "" } }
  ```

  DOĞRULA Ayardan sonraki ilk commit'te atıf satırı beklediğin gibi.

  Belge: Ayar referansı

16 Tasarım ve arayüz

Tasarımda en pahalı döngü "beğenmedim, tekrar dene". Onu kısaltan şey araştırma, birden çok yön ve görsel karşılaştırma.

16.1 Tasarımdan önce gerçek örneklerden ilham topla. İLERİ

NEDEN İlhamsız ilk taslak şablona yakın çıkar.

NASIL Kaynaklı bir ilham raporu iste.

```
Alışkanlık uygulamalarından 8 örnek bul, ana ekranlarını karşılaştır.
Her biri için: neyi iyi yapıyor, bizim uygulamaya ne alınabilir, kaynak.
```

DOĞRULA Her öneri bir gerçek örneğe dayanıyor mu?

İş akışı önerisi

16.2 İki-üç farklı yönü tam ekran çizdir. İLERİ

NEDEN Anlatılan tasarım kafada farklı canlanır, alt alanlar unutulur.

NASIL Her yönü alt alanlar dahil tam ekran iste.

DOĞRULA Seçimini görsele bakarak yapabiliyor musun?

İş akışı önerisi

16.3 Beğenmediğinde aynı yönü değil, farklı bir yönü iste. İLERİ

NEDEN Rötuş aynı fikrin varyasyonunu üretir.

NASIL Neyin çalışmadığını söyle ve yön değişikliği iste.

```
Bu yönden çık. Kart dizisi yerine tek bir odak alanı dene,
renk değil boşluk ve tipografiyle hiyerarşi kur.
```

DOĞRULA Yeni öneri ilkinden yapısal olarak farklı mı?

İş akışı önerisi

16.4 Tasarım dilini bir kez yaz, her turda yeniden anlatma. İLERİ

NEDEN Renk, tipografi ve boşluk kuralları her seferinde sohbetle aktarılınca kayar.

NASIL Projede tasarım ilkeleri dosyası ya da tasarım skill'i.

DOĞRULA Yeni oturumda tasarım dili sorulunca dosyadan cevap gelmeli.

İş akışı önerisi

17 Mobil ve simülatör

iOS ya da Android uygulaması geliştiriyorsan simülatör en çok kullanılan araç olur. Paylaşılan cihaz, elle tıklama ve açık kalan simülatör en sık zaman kaybı.

- [ ] **17.1 Paralel testlerde cihaz sahipliğini ve UDID'yi kaydet.** İLERİ

  NEDEN Paylaşılan simülatörde başka oturumun uygulaması öne gelir, senin uygulaman bozuk sanılır.

  NASIL Önce mevcut cihaz ve runtime listesini oku; gerekli ise göreve özel cihaz oluştur, dönen UDID'yi kaydet.

  ```
  xcrun simctl list devices available
  xcrun simctl list devicetypes
  xcrun simctl list runtimes
  # create için bu listelerdeki geçerli tip ve runtime kimliğini kullan.
  ```

  DOĞRULA Testin cihazı, runtime'ı ve uygulama build'i kayıtlı; başka görevin cihazı değiştirilmemiş.

  İş akışı önerisi

- [ ] **17.2 Tekrarlanan arayüz akışını otomatik teste çevir.** İLERİ

  NEDEN Elle tıklayarak doğrulama her seferinde aynı maliyeti öder.

  NASIL Maestro, XCUITest ya da Espresso akışı, tek komutla koşan.

  ```
  appId: com.ornek.fiyat
  ---
  - launchApp
  - tapOn: "Ürün ekle"
  - inputText: "Kahve"
  - assertVisible: "Kahve"
  ```

  DOĞRULA Akış bozuk bir ekranda kırmızı dönmeli.

  İş akışı önerisi

- [ ] **17.3 Türkçe metin girişini kullanılan araçta gerçekten dene.** İLERİ

  NEDEN Bazı otomasyon giriş yöntemleri Unicode veya klavye davranışını eksik taklit edebilir.

  NASIL Desteklenen giriş yolunu seç; İstanbul, ışık ve çığ gibi metinleri ekranda ve kaydedilen veride karşılaştır. ASCII yardımcı fixture gerçek dil testinin yerine geçmesin.

  DOĞRULA Girilen, gösterilen ve kaydedilen metin eşit; denenemeyen giriş yolu NOT_RUN.

  İş akışı önerisi

- [ ] **17.4 Farklı ekran boyutlarında kontrol et.** İLERİ

  NEDEN Küçük ekranda taşan metin büyük ekranda görünmez.

  NASIL En küçük ve en büyük desteklenen cihazda ekran görüntüsü.

  DOĞRULA İki görüntüde de taşma yok.

  İş akışı önerisi

- [ ] **17.5 Göreve ait geçici simülatörü kanıtları koruyup sil.** İLERİ

  NEDEN Shutdown cihazın veri alanını kaldırmaz; erase de cihaz kaydını silmez.

  NASIL Önce log ve ekran görüntülerini dışarı al, aktif kullanıcı olmadığını doğrula. Yalnız bu görevde oluşturduğun kesin UDID'yi kapat ve sil.

  ```
  xcrun simctl shutdown <GOREVE_AIT_UDID>
  xcrun simctl delete <GOREVE_AIT_UDID>
  xcrun simctl list devices --json
  ```

  DOĞRULA UDID cihaz listesinden kalkmış; önceden var olan veya sahibi belirsiz cihaz korunmuş.

  İş akışı önerisi

18 Bilgi birikimi

Öğrenilen ders yazılmazsa bir sonraki oturumda yeniden öğrenilir. Yanlış yere yazılırsa da kaybolur.

- [ ] **18.1 Dersi doğru katmana yaz.** İLERİ

  NEDEN Makine gerçeği bir proje dosyasında kalırsa diğer projelere görünmez.

  NASIL "Bu cümle nerede doğru olmaktan çıkıyor" diye sor.

  ```
  Makine ve araç gerçeği   -> ~/.claude/CLAUDE.md
  Koda ait gerçek          -> proje CLAUDE.md
  Repo + bu makine         -> CLAUDE.local.md
  Tarihli ölçüm, kanıt     -> ayrı kanıt dosyası
  ```

  DOĞRULA Her yeni kural doğru dosyada mı?

  Belge: Talimat ve bellek

- [ ] **18.2 "Sonra" ve "unutma" denen her şeyi hemen dosyaya yazdır.** İLERİ

  NEDEN Oturum içi hafıza devirde kaybolur.

  NASIL Proje kökünde bekleyenler dosyası, CLAUDE.md'de buna dair tek kural.

  DOĞRULA "Neler kaldı" sorusu dosyadan cevaplanıyor.

  Belge: Talimat ve bellek

- [ ] **18.3 Aynı düzeltme üçüncü kez geldiyse kural yap.** İLERİ

  NEDEN Üçüncü tekrar artık tesadüf değil.

  NASIL Düzeltmeyi say, üçte kurala çevir.

  DOĞRULA Son ayın düzeltmelerinden kaçı kurala dönüştü?

  Belge: Talimat ve bellek

- [ ] **18.4 Kuralı kısa, kanıtını ayrı tut.** İLERİ

  NEDEN Kısa kural bağlamda taşınır, uzun kanıt yalnız gerektiğinde açılır.

  NASIL Kuralın sonunda kanıt dosyasına bir atıf.

  DOĞRULA Her global kuralın bir kanıt atfı var mı?

  Belge: Talimat ve bellek

19 Bakım ritmi

Ayarlar, kurallar ve izin listeleri zamanla eskir. Küçük ve düzenli bakım, büyük temizlikten ucuzdur.

- [ ] **19.1 Haftalık: kullanımı ve bekleyenleri gözden geçir.** İLERİ

  NEDEN Limiti ve yarım işleri hafta bitmeden görürsün.

  NASIL `/usage` ve aktif projelerin bekleyenler dosyası.

  DOĞRULA Haftalık not: en çok harcayan proje, en eski bekleyen madde.

  Belge: Kullanım ve maliyet

- [ ] **19.2 Eski worktree ve disk kullanımını sahiplik bilgisiyle gözden geçir.** İLERİ

  NEDEN Disk doluluğu derleme ve testleri aksatabilir; tek bir yüzde bütün projeler için yeterli sınır değildir.

  NASIL Aktif süreç, yerel değişiklik ve kurtarma ihtiyacını kontrol et. `prune` yalnız eski yönetim kayıtlarını temizler; kullanılan checkout'u kaldırmaz.

  ```
  git worktree list
  git worktree prune --dry-run
  df -h .
  ```

  DOĞRULA Kaldırılan kesin yol ve korunmuş kanıt belli; boş alan işin beklenen derleme ihtiyacına yeterli.

  Belge: Git worktree

- [ ] **19.3 Aylık: CLAUDE.md'leri buda.** İLERİ

  NEDEN Kurallar birikir, bazıları artık hiç tetiklenmez.

  NASIL Geçerliliğini kaybeden kuralı düzelt; seyrek ama kritik olanı sırf geçen ay tetiklenmedi diye silme.

  DOĞRULA Çelişen veya eski kurallar düzeltilmiş; dosyanın küçük olması tek başına başarı ölçütü değil.

  Belge: Talimat ve bellek

- [ ] **19.4 Aylık: izin listelerini ve hook testlerini gözden geçir.** İLERİ

  NEDEN Yeni komutlar eklenir, Claude Code güncellemesi hook girdisini değiştirebilir.

  NASIL `/fewer-permission-prompts`, ardından hook testleri.

  DOĞRULA Gözetimsiz koşumlarda izin bekleyen adım yok, testler yeşil.

  Belge: Hook referansı

- [ ] **19.5 Çeyrekte bir: kendi kullanım analizini çıkar.** İLERİ

  NEDEN Alışkanlığın değişip değişmediğini sayı söyler.

  NASIL Claude'dan oturum kayıtlarını taramasını iste.

  ```
  ~/.claude/projects altındaki oturum kayıtlarını tara. Çıkar:
  mesaj sayısı, en sık kısa mesajlar ("devam" gibi), bağlam dolma sayısı,
  alt ajan çağrılarının model dağılımı. Önceki çeyrekle karşılaştır.
  ```

  DOĞRULA Sonuçları önceki çeyrekle karşılaştır.

  Belge: Kullanım ve maliyet

- [ ] **19.6 Sürüm, platform ve sağlayıcı farkını kaydet.** İLERİ

  NEDEN Belgede olan özellik eski CLI'da, başka sağlayıcıda veya masaüstü arayüzünde aynı olmayabilir.

  NASIL `claude --version`, `claude --help` ve ilgili komut menüsünü kontrol et; sürüm yükseltmesini kurulum yöntemine göre yap.

  DOĞRULA Rapor ortamı ve erişilemeyen özelliği belirtir; belge kontrolü canlı çalışma testi sayılmaz.

  Belge: Genel bakış

Kendi puan kartın

Her bölüm için kaç maddenin yapıldığını, kaçının sırada olduğunu yaz. Üç ay sonra yeniden doldur.

#	BÖLÜM	MADDE	YAPILDI	SIRADA	GEREK YOK
1	Talimat katmanları	7			
2	Bağlam yönetimi	6			
3	Görev verme	7			
4	Prompt yazımı: önce ve sonra	4			
5	Model, effort ve maliyet	5			
6	İzinler	7			
7	Güvenlik ve gizlilik	7			
8	Hook'lar	5			
9	Skill, komut, ajan, MCP	8			
10	Ekiple kullanım	3			
11	Paralel çalışma	6			
12	Doğrulama	7			
13	Gözetimsiz koşum	6			
14	Rutinler ve zamanlama	6			
15	Git ve yayın	4			
16	Tasarım ve arayüz	4			
17	Mobil ve simülatör	5			
18	Bilgi birikimi	4			
19	Bakım ritmi	6			
	Toplam	**107**			

21 Ekim 2026 · Kaynak kontrolü; yerel CLI 2.1.284. Özellikler hızlı değişiyor; bir komut ya da ayar adı çalışmıyorsa code.claude.com/docs adresindeki güncel belgeye bak.